

EXPERIMENT-11

Simulate Bankers Algorithm for Deadlock Avoidance.

AIM: A program to simulate the Bankers Algorithm for Deadlock Avoidance.

//Bankers algorithm for deadlock avoidance.

ALGORITHM:

1. Start the program.
2. Get the values of resources and processes.
3. Get the avail value.
4. After allocation find the need value.
5. Check whether its possible to allocate.
6. If it is possible then the system is in safe state.
7. Else system is not in safety state.
7. If the new request comes then check that the system is in safety.
8. or not if we allow the request.
9. stop the program.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int n,r,i,j,k,p,u=0,s=0,m;
int block[10],run[10],active[10],newreq[10];
int max[10][10],resalloc[10][10],resreq[10][10]; int totalloc[10],totext[10],simalloc[10];
clrscr();printf("Enter the no of processes:"); scanf("%d",&n);
printf("Enter the no of resource classes:"); scanf("%d",&r);
printf("Enter the total existed resource in each class:"); for(k=1;k<=r;k++)
scanf("%d",&totext[k]);
printf("Enter the allocated resources:"); for(i=1;i<=n;i++)
for(k=1;k<=r;k++) scanf("%d",&resalloc[k]);
printf("Enter the process making the new request:"); scanf("%d",&p);
printf("Enter the requested resource:"); for(k=1;k<=r;k++) scanf("%d",&newreq[k]);
printf("Enter the process which are n blocked or running:"); for(i=1;i<=n;i++)
{
if(i!=p)
{
printf("process %d:\n",i+1); scanf("%d%d",&block[i],&run[i]);
}
}
block[p]=0;run[p]=0;
for(k=1;k<=r;k++)
{
j=0;
for(i=1;i<=n;i++)
```

```

{
totalloc[k]=j+resalloc[i][k]; j=totalloc[k];
}
}
for(i=1;i<=n;i++)
{
if(block[i]==1||run[i]==1) active[i]=1;
else active[i]=0;
}
for(k=1;k<=r;k++)
{
resalloc[p][k]+=newreq[k]; totalloc[k]+=newreq[k];
}
for(k=1;k<=r;k++)
{
if(totext[k]-totalloc[k]<0)
{
u=1;break;
}
}
if(u==0)
{
for(k=1;k<=r;k++) simalloc[k]=totalloc[k]; for(s=1;s<=n;s++) for(i=1;i<=n;i++)
{
if(active[i]==1)
{ j=0;
for(k=1;k<=r;k++)
{
if((totext[k]-simalloc[k])<(max[i][k]-resalloc[i][k]))
{
j=1;break;
}
}
}
if(j==0)
{
active[i]=0; for(k=1;k<=r;k++) simalloc[k]=resalloc[i][k];
}
} m=0;
for(k=1;k<=r;k++) resreq[p][k]=newreq[k]; printf("Deadlock willn't occur");
}
else
{
for(k=1;k<=r;k++)
{
resalloc[p][k]=newreq[k]; totalloc[k]=newreq[k];
}
printf("Deadlock will occur");
}
}

```

```
}  
getch();  
}
```

OUTPUT:

Input:

Enter the no of resources: 4

Enter the no of resource classes: 3

Enter the total existed resources in each class: 3 2 2

Enter the allocated resources: 1 0 0 5 1 1 2 1 1 0 0 2

Enter the process making the new request: 2

Enter the processes which are n blocked or running:

Process 1: 1 2

Process 3: 1 0

Process 4: 1 0

Output:

Deadlock will occur

B)

Simulate Bankers Algorithm for Deadlock Prevention.

AIM: A program to simulate Bankers Algorithm for Deadlock Prevention. PROGRAM:

Algorithm:

1. Start
2. Attacking Mutex condition : never grant exclusive access. but this may not be possible for several resources.
3. Attacking preemption: not something you want to do.
4. Attacking hold and wait condition : make a process hold at the most 1 resource at a time. make all the requests at the beginning. All or nothing policy. If you feel, retry. eg. 2- phase locking
5. Attacking circular wait: Order all the resources. Make sure that the requests are issued in the correct order so that there are no cycles present in the resource graph. Resources numbered 1 ... n. Resources can be requested only in increasing order. ie. you cannot request a resource whose no is less than any you may be holding.
6. Stop

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int cl[10][10],al[10][10],av[10],i,j,k,m,n,c,ne[10][10],flag=0; clrscr();
printf("\nEnter the matrix"); scanf("%d %d",&m,&n); printf("\nEnter the claim matrix"); for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&cl[i][j]);
}
}
printf("\nEnter allocated matrix"); for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&al[i][j]);
}
}
printf("\nThe need matrix"); for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
ne[i][j]=cl[i][j]-al[i][j];
printf("\t%d",ne[i][j]);
}
printf("\n");
}
printf("\nEnter available matrix"); for(i=0;i<3;i++) scanf("%d",av[i]);
printf("Claim matrix:\n"); for(i=0;i<m;i++)
```

```

{
for(j=0;j<n;j++)
{
printf("\t%d",cl[i][j]);
}
printf("\n");
}
printf("\n allocated matrix:\n"); for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
printf("\t%d",al[i][j]);
}
printf("\n");
}
printf(" available matrix:\n"); for(i=0;i<3;i++)
{
printf("\t%d",av[i]);
}
for(k=0;k<m;k++) for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
if(av[j]>=ne[i][j]) flag=1;
else break;
if(flag==1 && j==n-1) goto a;
}
}
a: if(flag==0)
{
printf("unsafestate");
}
if(flag==1)
{
flag=0; for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
av[j]+=al[i][j];
al[i][j]=1;
}
}
printf("\n safe state");
for(i=0;i<n;i++)
printf("\t available matrix:%d",av[i]);
}
getch();
}

```

OUTPUT:**Input:**

Enter the claim matrix:3 2 2 6 1 3 3 1 4 4 2 2

Enter allocated matrix:1 0 0 5 1 1 2 1 1 0 0 2 The need matrix:

2 2 2

1 0 2

1 0 3

4 2 0

Enter available matrix1 1 2

Output:

Claim matrix:

3 2 2

6 1 3

3 1 4

4 2 2

Allocated matrix: 1 0 0

5 1 1

2 1 1

0 0 2

Available matrix:

1 1 2

Safe State

EXPERIMENT-12

Simulate all file allocation strategies

a) Sequential

b) Indexed

c) Linked

File name: SEQUENTIAL.C

ALGORITHM:

1. Start
2. Read the number of files
3. For each file, read the number of blocks required and the starting block of the file.
4. Allocate the blocks sequentially to the file from the starting block.
5. Display the file name, starting block , and the blocks occupied by the file.
6. stop

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
struct sequence
{
char n[20]; int i;
}
s[20];
int create(int);
int del(int); void display(int); void main()
{
int x=0,j=0;
clrscr(); while(1)
{
printf("1.creation\n2.delete\n3.display\n4.exit"); printf("\nenter one option");
scanf("%d",&x); switch(x)
{
case 1:
j=create(j); break;
case 2: j=del(j);
//Computer Networks & Operating Systems Lab Manual break;
case 3: display(j); break;
case 4: exit(1);
default : printf("wrong option");
}
}
}
int create(int j)
```

```

{
int m,v; j++;
w:printf("\nenter the filename:"); scanf("%s",&s[j].n);
m=1;
while(m<j)
{
v=strcmp(s[j].n,s[m].n); if(v==0)
{
printf("file is already exist\nplease enter another name"); goto w;
} m++;
}
printf("\nenter field:"); scanf("%d",&s[j].i); return(j);
}
int del(int j)
{
j--;
return(j);
}
void display(int j)
{
int l; printf("filename\tfield"); for(l=1;l<=j;l++)
printf("\n%s\t\t%d\n",s[l].n,s[l].i);
}

```

Output:

```

1.creation
2.delete
3.display
4.exit
enter one option1
enter the file name:1.c
enter field:1
1.creation
2.delete
3.display
4.exit
enter one option1 enter the file name:2.c
enter field:2
1.creation
2.delete
3.display
4.exit
enter one option3
filename field
1.c 1
2.c 2 1.creation 2.delete 3.display 4.exit
enter one option4

```


(b) File name: INDEXED.C

ALGORITHM:

1. Start
2. Read the number of files
3. Read the index block for each file.
4. Link all the blocks of the file to the index block.
5. Display the file name, index block , and the blocks occupied by the file.
6. stop

PORGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
struct file
{
char n[20]; int fld,ind;
}s[20];
int no,i=-1,a,b,f,j=-1,fe,t; char tem[20]; void create();
void display(); void del(); void main()
{
clrscr(); while(1)
{
printf("\n\nmenu"); printf("\n1.create\n2.display\n3.delete\n4.exit"); printf("enter ur choice:");
scanf("%d",&no);
switch(no)
{
case 1: create(); break;
case 2: display(); break;
case 3: del(); break;
case 4: exit(0);
default: printf("wrong choice");
}
}
}
void create()
{ i++;
printf("\nenter the name of the recoed:"); scanf("%s",&s[i].n);
printf("\nenter the index no:"); scanf("%d",&s[i].ind); printf("\nenter the field no:"); scanf("%d",&s[i].fld);
j++;
}
void display()
{
for(a=0;a<i;a++)
{
for(b=0;b<j;b++)
if(s[b].ind>s[b+1].ind)
{
t=s[b].ind; s[b].ind=s[b+1].ind; s[b+1].ind=t; strcpy(tem,s[b].n); strcpy(s[b].n,s[b+1].n); strcpy(s[b+1].n,tem);
t=s[b].fld; s[b].fld=s[b+1].fld; s[b+1].fld=t;
}
```

```

}
else continue;
}
}
printf("\n      ");
printf("\n\t Index Recordname FieldNo"); for(i=0;i<=j;i++)
{
printf("\n\t%d\t",s[i].ind);
printf("\t%s",s[i].n);
printf("\t%d",s[i].fld);
}
i--;
printf("\n      \n");
}
void del()
{
int de,index=-1,k=0,l; if(i!=-1)
{
printf("enter index no to be deleted"); scanf("%d",&de);
index=de; while(s[k].ind!=de)
{ k++;
printf("\n\t\t\t%d",k);
}
for(l=k;l<=j;l++) s[l]=s[l+1];
i--;
j--;
printf("\nindex no %d file is deleted",index);
}
}

```

Output:

```

menu 1.create 2.display 3.delete
4.exitenter ur choice:1
enter the name of the recoed:a.java enter the index no:0
enter the field no:1 menu
1.create 2.display 3.delete
4.exitenter ur choice:1
enter the name of the recoed:b.c enter the index no:1
enter the field no:2 menu
    1.create
    2.display
    3.delete
    4.exitenter
ur choice:2
Index Recordname FieldNo 0 a.java 1
1 b.c 2
menu 1.create 2.display 3.delete
4.exitenter ur choice:4

```

c) File name: LINKED.C ALGORITHM:

1. Start
2. Read the number of files
3. For each file, read the file name, starting block, number of blocks and block numbers of the file. 4.
Start from the starting block and link each block of the file to the next block in a linked list fashion.
5. Display the file name, starting block, size of the file , and the blocks occupied by the file.
6. stop

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
typedef struct
{
int
bno,flag,next; }block; block b[200],b1;
void main()
{
int rnum();
int i,n,s,s1,p[30],r,k[20]; clrscr();
printf("\nEnter number of programs:"); scanf("%d",&n);
printf("\nEnter the memory block request"); for(i=1;i<=n;i++)
{
printf("\nEnter program requirement"); scanf("%d",&p[i]);
}
for(i=1;i<=n;i++)
{
s=rnum();
b[s].bno=0;
b[s].flag=1;
k[i]=0;
r=p[i]-1; while(r!=0)
{
s1=rnum(); b[s].next=s1;
b[s1].flag=1;
b[s1].bno=0; s=s1;
r=r-1;
}
b[s1].next=NULL;
}
printf("\n Starting blocks for program"); for(i=1;i<=n;i++) printf("\n%5d%5d",i,k[i]);
printf("\n allocated blocks"); for(i=1;i<=200;i++)
{
if(b[i].flag==1) printf("\n%5d%5d",b[i].bno,b[i].next);
}
}
```

```
int rnum()
{
int k,i; for(i=1;i<=200;i++)
{
}
return k;
}
```

Output:

```
Enter number of programs:2
Enter the memory block request
Enter program requirement3
Enter program requirement4
Starting blocks for program
1 0
2 0
allocated blocks 0 117
0 560 195
0 182
0 130
0 0
0 0
```

EXPERIMENT-13

DOWNLOAD AND INSTALL NACHOS OPERATING SYSTEM AND EXPERIMENT WITH IT

Step 1: Install Dependencies

Before installing NACHOS, you need tools like gcc, g++, make, and binutils. Run the following commands in **Ubuntu/Linux**:

```
sh
CopyEdit
sudo apt update
sudo apt install build-essential gcc g++ make
```

Step 2: Download NACHOS

Clone the NACHOS source code from GitHub:

```
sh
CopyEdit
git clone https://github.com/DrMonge/Nachos-Operating-System.git nachos
cd nachos/code
```

Step 3: Compile NACHOS

```
sh
CopyEdit
cd threads
make clean
make
```

If the build succeeds, NACHOS is ready!

PROGRAM:

```
#include "system.h"

void PrintThreadID(int id) {
    printf("Thread %d is running.\n", id);
    currentThread->Yield(); // Allow other threads to execute
    printf("Thread %d finished execution.\n", id);
}

void ThreadTest() {
    for (int i = 1; i <= 3; i++) {
        Thread *t = new Thread("TestThread");
        t->Fork((VoidFunctionPtr)PrintThreadID, (void*)i);
    }
}
```

OUTPUT:

```
Thread 1 is running.
Thread 2 is running.
Thread 3 is running.
Thread 1 finished execution.
Thread 2 finished execution.
Thread 3 finished execution.
```